Quiz # 1 (Data Structures) **BS DS Fall 2025 Afternoon**
Name:_____ **Roll number:**_____
Time Allowed: 25 min

Question#1 (5+5=10 Marks)---------- CLO3

Given the following 5 × 6 matrix, 0 5 0 0 8 0 2 0 0 7 0 0 0 0 3 0 0 6 0 4 0 0 9 0 1 0 0 2 0 0 Given the following 5 × 6 matrix, convert it into CSC representation. Find: 1. data[] 2. rowIndex[] 3. colPtr[]	data[] = {2, 1, 5, 4, 3, 7, 2, 8, 9, 6} rowIndex[] = {1, 4, 0, 3, 2, 1, 4, 0, 3, 2} colPtr[] = {0, 2, 4, 5, 7, 9, 10}
Consider a array: int A[4][5]; The base address of A[0][0] is 1000 and each integer occupies 4 bytes. Using row-major representation, calculate the address of: A[3][2]	Address(A[i][j])=Base+[(i×columns)+j]×size Substitute: $= 1000 + [(3 \times 5) + 2] \times 4$ $= 1000 + [15 + 2] \times 4$ $= 1000 + 68$ $\boxed{Address(A[3][2]) = 1068}$ **Answer: 1068**

Question#2 (10 Marks)---------- CLO3
You are provided with the implementation of a Polynomial class that represents a polynomial equation. Assume that all member functions are already implemented. Your task is to write the definition of the highlighted member function only.

```
class Term {
public:
   double coefficient;
   int power;
   Term(double c = 0, int p = 0);
};
class Polynomial {
private:
   Term* terms;      // dynamic array of terms
   int size;         // number of terms
   int capacity;     // capacity of dynamic array
public:
   Polynomial(int cap = 10);          // default constructor
   Polynomial(const Polynomial &other);   // copy constructor
   Polynomial& operator=(const Polynomial &other); // assignment
  Polynomial  operator*(const Polynomial &other);
   ~Polynomial();                  // destructor
   void addTerm(double coefficient, int power);
   int getDegree() const;
   double getCoefficient(int power) const;
   void addToCoefficient(double coefficient, int power);
    void setCoefficient(double newCoefficient, int power);
```

```
    double operator()(double value) const; // evaluate polynomial
};
```

```
Polynomial Polynomial::operator*(const Polynomial &other) const {
   Polynomial result;
   for (int i = 0; i < size; i++) {
      for (int j = 0; j < other.size; j++) {
         result.addTerm(terms[i].coefficient * other.terms[j].coefficient,
                  terms[i].power + other.terms[j].power);
      }
   }
   return result;
}
```